

Witness-CL: Counterexample-Preserving Online Skill Compilation for Continual Agents

Samuel Mausberg

AI-assisted research draft; author review and independent replication pending

Abstract

We study agents that improve from deployment experience without an intervening offline retraining stage. We propose Witness-CL, a research architecture separating two kinds of admission: exact, scoped programs justified by a complete hypothesis class, and broader candidate systems tested on fresh paired episodes. Its exact path preserves a subset of observations that reproduces the full-history version space, retaining counterexamples instead of repeatedly summarizing advice. Elementary conditional proofs establish truth retention, persistent unanimous predictions, exact witness condensation and a finite mistake bound for symbolic voting. A CPU reference study across five scenarios and 20 seeds shows novel-input transfer and equal accuracy to full-history symbolic induction, not superiority to language-model in-context learning. Negative controls expose incorrect conditional certificates under hidden drift and class misspecification. An audit simulation shows that modest improvements may require more fresh episodes than a short deployment provides. The artifact contains tests, raw records, a local model experiment harness and uncompiled Lean proof attempts. Public-benchmark superiority, general no-forgetting and novelty of the overall combination remain unestablished.

1 Problem and evidence

A useful continual agent should turn earlier experience into improved decisions on later, non-identical problems. Merely appending a transcript, retrieving a similar episode, or retaining model weights does not establish that outcome. The question here is whether reusable behavior can be learned and admitted online while measuring interference, cost and the conditions under which old behavior remains valid.

CL-Bench directly tests learning in stateful environments; its reported aggregate results make raw in-context learning (ICL) a necessary control, not a weak baseline [1]. The finding is not that every memory architecture loses on every task. AgentCL additionally uses controlled compositional streams to distinguish reuse of earlier sub-solutions from ordinary task competence [2]. Together they motivate a narrower unresolved target: a repeatable reward, transfer and retention advantage under a

matched backbone and matched total resource budget.

Interpretation of the requirements. The deployed system may update external state after each episode, and may perform online computation during those updates. The principal method holds neural parameters fixed. “Without offline retraining” excludes a separate post-deployment training phase, not online learning. “Without forgetting” is separated into retained bytes, retained parameters, retained behavior on protected inputs, and retained expected competence on a task distribution. None is silently substituted for another. We prove only conditional behavior retention for the exact branch; we formulate a weaker statistical claim for empirical changes.

Research hypothesis. The proposed advantage is that *decision-discriminating evidence* is a better unit of memory than a repeatedly rewritten natural-language conclusion. Where a sound model class is available, preserve exactly the observations needed to rule out competing behavior and compile the resulting rule. Where it is unavailable, maintain an empirical candidate rather than manufacture a logical certificate. The hypothesis is falsifiable: matched-budget raw ICL may remain better, and the delivered stress tests already refute an unconditional claim for the exact-only implementation.

2 Position relative to prior work

Memory and adaptation. ACE evolves structured playbooks with generation, reflection and curation, including online adaptation [3]. MemProbe separates interaction, insight and skill memory and filters unreliable consolidation [2]. Their successes make “structured memory” insufficient as a novelty claim. Inference from their design, rather than an asserted experimental result, is that a plausibility or syntactic check does not automatically certify future policy improvement.

Voyager already stores reusable executable skills and learns from environment interaction without updating its underlying language model [4]. EvoTest evolves a broader agent configuration across episodes, and TTHE adapts executable harnesses at test time using execution-derived proxy signals [5, 6]. These are direct competitors

to a general self-improving harness, not peripheral memory baselines. Their existence rules out claiming the first deployed system that improves without gradients. The remaining question is reliable transfer and retention across regimes and budgets, not whether online improvement has ever occurred.

ALMA meta-learns executable memory designs [7]; predeployment meta-learning is not itself incompatible with avoiding offline retraining after deployment. User as Code further makes executable memory and an append-only evidence architecture prior art [8]. Decision-Aware Memory Cards ranks context by decision-oriented utility and compresses it into typed evidence; its reviewed version explicitly separates counterfactual-inspired scores from identified randomized effects [9]. Witness-CL therefore cannot claim that code, ledgers, typed cards or outcome-aware retrieval are new.

Learning theory and synthesis. The exact branch is grounded in version-space reasoning and the KWIK principle of withholding confident predictions when evidence is insufficient [10]. Counterexample-guided synthesis and representative-example selection also have substantial history [11, 12]. The condensation proof below is an elementary invariant, not a newly discovered general learning theorem. Conservative bandits study baseline-relative learning under structural assumptions [13]; time-uniform inference supplies tools for valid adaptive admission [14]. The proposed contribution to test is their operational combination with scoped behavioral compilation and explicit episode provenance.

Weight-based alternatives. Regularization, episodic replay constraints and parameter isolation address catastrophic interference through different mechanisms [15–17]. End-to-end test-time training is another relevant adaptation direction [18]. These methods should not be dismissed because the main design freezes weights. However, a frozen old parameter block does not, by itself, prove that a changing router or context continues to invoke the same behavior. The artifact includes an ordinary online regression baseline; a neural adapter extension is specified but not implemented.

3 Exact online compilation

3.1 Setting and assumptions

An episode supplies observable scope/version s_t , input x_t , an action $a_t \in \mathcal{A}$, and subsequent success feedback $r_t \in \{0, 1\}$. For each scope s , fix a finite class $\mathcal{H}_s$ of total deterministic functions from inputs to actions. The exact theorem assumes a stationary target $h_s^* \in \mathcal{H}_s$, with trusted feedback $r_t = \mathbf{1}\{a_t = h_s^*(x_t)\}$. Public scope routing must be correct. A schema version can be public; an evaluator’s hidden regime label is not.

The reference grammar is deliberately small:

$$h_{a,b}(x) = (ax + b) \bmod p, \quad a, b \in \{0, \dots, p-1\}. \quad (1)$$

For the executed study $p = 7$. Every inductive control receives the same grammar. The learner does not receive hidden coefficients or the correct answer following a failed action. General language-agent tasks do not automatically satisfy these assumptions. In particular, agreement among an LLM’s sampled programs is not agreement over an exhaustive, realizable class.

3.2 Version spaces and witnesses

For feedback record $e = (x, a, r)$ define

$$C_e(h) \equiv (\mathbf{1}\{h(x) = a\} = r). \quad (2)$$

Within each public scope, update

$$\mathcal{V}_0 = \mathcal{H}, \quad \mathcal{V}_t = \{h \in \mathcal{V}_{t-1} : C_{e_t}(h)\}. \quad (3)$$

A failure excludes hypotheses predicting the failed action. It does not reveal which of the other actions is correct. Empty sets trigger quarantine, never vacuous unanimity or an automatic reset into a supposed proof.

The active witness set retains an observation only when the live set strictly shrinks:

$$\mathcal{W}_t = \begin{cases} \mathcal{W}_{t-1} \cup \{e_t\}, & \mathcal{V}_t \neq \mathcal{V}_{t-1}, \\ \mathcal{W}_{t-1}, & \text{otherwise.} \end{cases} \quad (4)$$

The complete raw history remains available for auditing and empirical fallback. A witness is thus an exact-class compression object, not an assertion that the rest of the transcript lacks value to a language model.

Definition 1 (Conditional certificate). *An action a at input x is certified by $\mathcal{V}$ exactly when $\mathcal{V} \neq \emptyset$ and $h(x) = a$ for every $h \in \mathcal{V}$.*

Certificates carry the public scope, input, action, generation and live-class identity. Other public scopes retain their own state. If no certificate applies, the symbolic reference uses majority vote with deterministic tie breaking. The optional model experiment instead calls a frozen model on the available history. The statistical and mistake-bound implications of those fallbacks differ.

3.3 Conditional guarantees

Theorem 2 (Truth and contract preservation). *Under the assumptions above, $h^* \in \mathcal{V}_t$ for all t . Every certified prediction is correct, and a certified prediction remains certified after any subsequent update in the same unchanged scope.*

Proof. Initially $h^* \in \mathcal{H}$. Trusted feedback makes $C_{e_t}(h^*)$ true, so induction through Eq. (3) retains h^* . A unanimous prediction equals the prediction of h^* . Each future

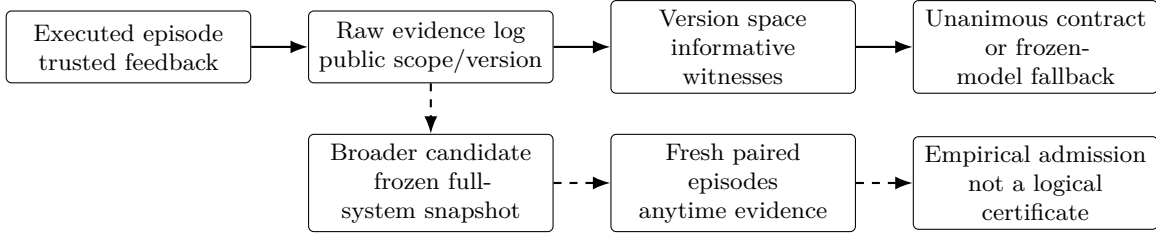


Figure 1: The two admission paths deliberately have different guarantees. Solid arrows are the implemented finite-class reference. The empirical auditor is implemented and tested as a primitive, but the dashed general-agent integration and public-benchmark compiler are proposed, not executed.

Algorithm 1: Exact path for one observed public scope

Input: Complete finite class $\mathcal{H}$; fallback π_0
 $\mathcal{V} \leftarrow \mathcal{H}$; $\mathcal{W} \leftarrow \emptyset$; $L \leftarrow \emptyset$;
for each episode (x, s) **do**
 if scope valid, $\mathcal{V} \neq \emptyset$, all $h \in \mathcal{V}$ agree at x **then**
 execute the unanimous action; cache its conditional contract;
 else
 execute fallback; do not label it certified;
 receive actual action-success feedback e ; append e to L ;
 $\mathcal{V}' \leftarrow \{h \in \mathcal{V} : C_e(h)\}$;
 if $\mathcal{V}' \neq \mathcal{V}$ **then**
 append e to $\mathcal{W}$;
 $\mathcal{V} \leftarrow \mathcal{V}'$;
 if $\mathcal{V} = \emptyset$ **then**
 quarantine this scope; invalidate its contracts;

version space is a nonempty subset of the previous one, so unanimity persists. This also shows why nonemptiness cannot be dropped from the certificate definition. $\square$

Theorem 3 (Exact witness condensation). *Replaying only $\mathcal{W}_t$ from $\mathcal{H}$ yields exactly $\mathcal{V}_t$. Moreover, $|\mathcal{W}_t| \leq |\mathcal{H}| - |\mathcal{V}_t|$; under realizability, $|\mathcal{W}_t| \leq |\mathcal{H}| - 1$.*

Proof. An omitted constraint was satisfied by every live hypothesis when it arrived. Every later live set is a subset, so the constraint remains redundant. Induction over retained and omitted constraints gives exact equality with full-history filtering. Every retained observation removes at least one previously live hypothesis, and no removed hypothesis returns. Summing these decreases gives the cardinality bound. Truth retention implies $|\mathcal{V}_t| \geq 1$ under realizability. $\square$

This is not a minimal witness-set algorithm: later evidence can make an earlier retained witness redundant. Optional deletion would require checking equal live sets and paying for that computation. The bound counts observations, not the bit length of programs, and it does not make the raw evidence ledger bounded.

Proposition 4 (Voting-policy mistake bound). *Let $m = |\mathcal{H}|$ and $k = |\mathcal{A}| \geq 2$. If uncertified decisions use a most frequent live prediction, the number M of mistakes in one stationary realizable scope satisfies*

$$M \leq \frac{\log m}{-\log(1 - 1/k)}. \quad (5)$$

Proof. A most frequent prediction is shared by at least $|\mathcal{V}|/k$ hypotheses. A mistake eliminates all of them. Other observations cannot enlarge the set. Consequently $1 \leq |\mathcal{V}_t| \leq m(1 - 1/k)^M$, and taking logarithms gives the result. Sum independently over public scopes. $\square$

The bound is an elementary elimination argument and is not claimed as novel. It does not apply to an arbitrary LLM fallback, which can select an unsupported action and learn nothing from its failure. Nor does it guarantee identification from an uninformative input sequence.

Corollary 5 (Transfer in the affine reference). *For prime p , two successful observations at distinct inputs uniquely identify an affine hypothesis in $\mathbb{F}_p$, allowing prediction at unobserved inputs.*

Proof. If the successes are (x_1, y_1) and (x_2, y_2) , then $a = (y_2 - y_1)(x_2 - x_1)^{-1}$ and $b = y_1 - ax_1$. The inverse exists because $x_2 - x_1 \neq 0$ in the field. The simulator supplies no successful observations for free; they must arise from the agent’s executed actions. $\square$

3.4 What the guarantees do not mean

Correct stored contracts can remain unused because of a bad router, or be applied to the wrong environment. Parameter immutability does not rule out those errors. The present implementation uses public keys rather than learning a hidden task identifier, and tests unrelated-key noninterference. Hidden regime changes, incorrect abstractions and noisy rewards violate the exact theorem. The negative experiments make those violations observable rather than interpreting quarantine as prevention of all bad actions.

A perfect full-history symbolic learner can reconstruct precisely the same version space and witnesses. No information-theoretic advantage over that control is

claimed. Any advantage is due to computational amortization, context use, or more reliable decision execution, all of which require measurement with real model serving, including prefix caching and compiler overhead.

4 Empirical admission of broader skills

4.1 Fresh evidence rather than self-approval

For a candidate outside the exact regime, freeze its entire system snapshot and that of the current incumbent before auditing. Snapshot identity includes code, model version, prompts, memory, dependencies, route and within-episode procedure. Record its birth episode. Proposal data may generate a candidate but cannot be reused as its fresh validation stream.

Assign a globally unique index $j \geq 1$ and budget

$$\delta_j = \frac{\delta}{j(j+1)}, \quad \sum_{j=1}^{\infty} \delta_j = \delta. \quad (6)$$

On fresh paired episodes measure $D_i = R_i(\pi_j) - R_i(\pi_{0,j}) \in [-1, 1]$. Pairing requires honest simulator forks or isolated test environments. Both branches count as interactions and compute. A saved trajectory does not reveal outcomes for actions never executed. For non-cloneable deployments, a randomized-assignment design is needed; the corresponding estimator and integration are future work.

4.2 An anytime test

For the fixed stake set $\Lambda = \{1/10, 1/4, 1/2, 3/4, 1\}$, define

$$E_{j,n} = \frac{1}{|\Lambda|} \sum_{\lambda \in \Lambda} \prod_{i=1}^n (1 + \lambda D_i). \quad (7)$$

The reference uses rational arithmetic and admits when the running process first crosses $1/\delta_j$. There is no optional-stopping exemption, repeated reset or unreported reuse of the same candidate index.

Theorem 6 (Conditional false-admission control). *Suppose each null candidate’s fresh differences satisfy $\mathbb{E}[D_i | \mathcal{F}_{i-1}] \leq 0$, conditional on the history at its registration. With the frozen snapshots and unique allocation above, the probability of ever admitting any such null candidate is at most δ .*

Proof. Every factor in Eq. (7) is nonnegative. For a fixed stake, the product has conditional next-step expectation at most its current value, since $1 + \lambda \mathbb{E}[D_i | \mathcal{F}_{i-1}] \leq 1$. Each product, and their mixture, is therefore a nonnegative supermartingale starting at one. Ville’s inequality bounds its probability of ever crossing $1/\delta_j$ by δ_j . The

bound holds conditional on each candidate’s adaptive pre-audit history. A union bound and Eq. (6) give the result. These are standard sequential-inference arguments [14], not a new concentration result. $\square$

For stationary iid scoped tasks, the null is nonpositive mean benefit. The conclusion is not pointwise non-degradation, future robustness to drift, or a safety theorem about objectives not represented by reward. If an incumbent changes during an audit, passing against the old snapshot is insufficient. The primitive’s promotion gate checks current incumbent identity, but a trusted harness must compute real snapshot hashes and maintain globally unique indices across restarts. Those operational assumptions are not verified by supplying matching strings.

If each admitted change affects only its guard’s scope and all scoped task distributions remain stationary, expected scoped reward improves on the event of no false admission. Untouched scopes retain their policies. This conditional corollary does not merge with the exact theorem unless all exact-path assumptions are also true, and it does not guarantee a beneficial change will ever be admitted.

5 Implementation and measured mechanisms

5.1 What was executed

The repository implements the finite DSL, binary-feedback elimination, witness condensation, certificate caching, scope quarantine, a single-writer hash-linked ledger, a rational paired auditor and an online regression control. The optional chat-completions client is mock-tested and records prompts, responses, usage and invalid outputs. No actual model request, GPU run or public-benchmark evaluation was executed. The empirical auditor is not yet connected to a general-agent candidate generator or a native CL-Bench adapter.

The synthetic study evaluates six symbolic methods over five scenarios, 20 seeds and 384 episodes per run. Four publicly identified systems recur in 12-episode blocks. The learners see their input, action and binary success, not the scoring target. Methods are a stateless constant, bounded exact-input lookup, unbounded lookup, 24-observation windowed induction, full-history replay induction, and Witness. Full replay rebuilds the same class from all previous observations for each decision, with identical tie breaking.

Scenarios are stationary interleaving, new inputs after the halfway point, a publicly versioned affine change, the same change without a public version update, and quadratic targets outside the affine class. The last case first exposes only inputs zero and one, where the wrong linear explanation fits, then tests new inputs. These are

deliberately diagnostic constructed streams, not expert-validated language-agent tasks. Confidence intervals re-sample independent run seeds, not individual correlated episodes.

5.2 Transfer, retention and computation

Table 1 shows that the inductive policy transfers beyond recorded inputs, whereas exact-input lookup must gather new evidence. Equality with full symbolic replay is the expected result: compilation does not create information. The stationary streams contain zero incorrect conditional contracts. Tests also check that updates in another scope leave existing contracts intact.

Across a stationary 384-episode run, Witness retains an average 29.4 active witness observations across all scopes, representing the same live class as the full record. This is a 13.06-fold observation-count reduction for that constructed setting. It still stores the complete raw history. It evaluates hypotheses 2,340.9 times on average, versus 78,146.1 for full symbolic reconstruction, a factor of 33.38. This deliberately redundant reconstruction is an information control, not a competitive serving kernel; its operation-count ratio is not an inference latency or GPU speedup.

A decisive failure. The nonlinear test yields late reward 14.09% for Witness versus 68.12% for full lookup. Four wrong conditional certificates occur per seed, one per scope, before contradictions trigger quarantine. Hidden drift likewise causes wrong contracts before detection. Consequently the current exact-only prototype is *already falsified* as a universal solution. The key unsolved bridge is obtaining useful, sound abstractions and recognizing invalid applicability, not merely making the version-space update faster.

5.3 Audit cost as a second failure mode

We simulate 2,000 independent audit streams for each true mean difference, with $D \in \{-1, 0, 1\}$, disagreement probability 0.3, and fixed first-candidate budget $\delta_1 = 0.025$. This is a floating log-space power diagnostic for the same betting mixture, not a simulation of an LLM. Unit tests use exact rational arithmetic and include exhaustive finite null paths as a separate check.

For a true five-percentage-point gain, only 6.4% pass by 128 paired episodes; 22.7% pass by 512. Small effects or short-lived rules can therefore be practically unlearnable through this conservative admission branch at the available budget. The zero-mean row crosses in 1.95% of runs by 2,048 episodes, consistent with, but not proving, the 2.5% null crossing bound. Disagreement, effect size, registration index and the stake mixture all influence power. Stronger adaptive betting or variance-sensitive tests are possible, but cannot create unobserved information or remove the cost of honest audit feedback.

6 Experiment that would decide the proposal

Primary test. Freeze the method, code, model identifiers and evaluation protocol before the public run. Compare raw ICL, Mem0, ACE, MemProbe and TTHe or EvoTest, then the exact and combined Witness variants, using matched backbones. Run native CL-Bench reward and gain, with a controlled-composition transfer evaluation as an independent diagnostic. Use real upstream implementations rather than assign published names to toy heuristics. Inspect feedback availability and scope metadata for every task; never provide hidden answer fields to the learner.

The proposed success threshold is at least five percentage points of aggregate normalized reward over the best prespecified baseline, improvement on at least four of six native tasks, and a paired 95% interval excluding zero. These are research targets, not power-guaranteed predictions. A separate pilot should estimate variance; ten seeds alone is not a statistical justification.

Retention and leakage. Reserve an authorized held-out probe distribution for each learned scope and compare frozen snapshots after intervening scopes. Probe outcomes cannot enter memory or candidate selection. Report worst-family degradation as well as mean backward transfer. A proposed noninferiority tolerance is one normalized percentage point with prespecified one-sided inference and multiplicity handling. Track unseen-input and unseen-composition transfer separately from repeated queries. Under exact assumptions, any wrong contract fails the correctness test.

Costs and ablations. Report both iso-feedback and iso-total-cost comparisons, charging all synthesis, audit branches, token usage, storage and latency. Measure learning-curve area, native reward/gain, memory bytes, certificate coverage, fallback frequency, audit decisions and time to useful behavior. Ablate negative evidence, witness condensation, scope guards, exact execution and fresh admission. The witness-only prompt ablation is important: equal symbolic version spaces do not prove equal behavior from a finite language model given compressed text.

Resources. The present reference needs a CPU. The model pilot needs one served frozen model, an inference device with sufficient weight and context memory, and permitted task environments. A GH200 is a plausible platform in the surrounding project, not a measured speed claim. BF16 weight storage alone is approximately $2P$ bytes for P parameters; KV state, activations and runtime memory must be added. No neural optimizer state is needed for the primary method. Large-scale

Scenario	Overall reward (%)			Late-half reward (%)		
	Window	Lookup	Witness / full replay	Witness	Lookup	Wrong certs.
Stationary interleaving	52.84	78.12	94.17 [93.48, 94.82]	100.00	98.26	0.0
Previously unseen inputs	51.68	78.19	93.83 [93.33, 94.36]	100.00	68.72	0.0
Public version change	51.50	58.33	88.27 [87.34, 89.14]	88.20	58.67	0.0
Hidden rule change	51.50	42.89	52.12 [50.57, 53.87]	15.91	27.79	4.0
Misspecified class	37.99	77.89	50.87 [49.09, 52.79]	14.09	68.12	4.0

Table 1: Executed symbolic CPU results. Window means full induction from the most recent 24 global observations; lookup retains the entire exact-input history. Full-history symbolic induction and Witness have identical accuracy. Brackets are 95% bootstrap intervals across 20 seeds. Wrong certificates are counts per 384-episode run. The negative rows explicitly violate the exact theorem’s assumptions. None of these numbers is an LLM ICL or CL-Bench score.

True gain	By 128	By 512	By 2,048
-10 pp	0.10%	0.10%	0.10%
+0 pp	1.25%	1.60%	1.95%
+2 pp	2.35%	4.50%	12.10%
+5 pp	6.40%	22.70%	85.90%
+10 pp	25.85%	88.60%	100.00%
+20 pp	93.75%	100.00%	100.00%

Table 2: Simulated admission fractions by number of fresh paired episodes, with 2,000 runs per row. Positive mean benefit is not enough for quick admission. These percentages are synthetic power estimates, not proof of the probability theorem or observed agent performance.

models may instead require an authorized API budget, with actual token accounting.

Let S be total episodes in the pinned native schedules, M the model count, A the arm count and R the run count. Stateful plus corresponding stateless runs cost about $2MARS$ episode-agent evaluations before audit branches and ablations. For two models, seven arms and ten runs, this is $280S$, not a single cheap benchmark invocation. Begin with a small frozen-model pilot and three task families before funding the complete study. Exact wall-clock or dollar estimates require a selected model, endpoint and measured throughput.

Kill criteria. Reject the main claim if matched-budget ICL remains better, gains disappear on novel inputs, hidden changes remain poorly handled, or average gains mask protected scope regressions. Reject the empirical branch for a regime if most useful candidates cannot pass within their relevance horizon. An improvement bought by extra hidden simulator calls is a resource tradeoff, not matched-budget learning. A sampled candidate pool masquerading as a complete hypothesis class invalidates the logical claim. Failure of Lean compilation invalidates any kernel-verification claim, independently of empirical results.

7 Training extension and deployment safety

The primary system changes external learned state, not neural parameters. An optional recursive least-squares control updates one scoped predictor from each actual scalar label; tests compare it to the corresponding batch solution. That batch computation is only a unit-test oracle, not a deployment training phase. A neural extension could freeze old adapters and learn a new scoped residual from genuine labels or verified outcomes. It remains unimplemented, and it would still require behavioral routing and retention evaluation.

The shipped arithmetic DSL cannot run arbitrary supplied code, access files or call tools. A general compiler must preserve existing permission checks outside learned state. Outcome feedback needs trustworthy provenance; self-critique or user instructions are not substitutes. A hash chain is useful for detecting partial log edits with a trusted external head, but does not authenticate sources or resist rewriting both the log and its trusted reference.

Post-hoc quarantine does not prevent the first dangerous action after a hidden change. Irreversible actions require independent current validation or a safe abstention mechanism. An average-reward e-process does not certify all safety constraints. Privacy also takes precedence over indefinite retention: authorized deletion must invalidate dependent evidence and compiled behavior. The reference does not implement a complete deletion or unlearning system.

8 Formalization and artifact status

The repository supplies 13 Lean theorem attempts for the abstract logical core, including truth retention, conditional soundness, refinement preservation, certificate equivalence, persistent redundancy, sequential refinement and other-scope noninterference. No `sorry`, `admit` or custom axiom is provided in the source. The authoring environment lacked Lean and the official toolchain down-

load was blocked; `lake build` exited with command-not-found. **These sources are not kernel-checked.** The probability theorem, cardinality/mistake bounds, field argument and refinement to Python are not formalized. Mathematical proofs and executable tests must not be relabeled as formal verification.

Python tests, raw synthetic records and generated tables are included, together with the exact environment description and an optional model-facing runner. Public-benchmark integration remains a documented boundary: the upstream ICL interface was inspected, but task-specific compilers and native execution were not completed. The paper is a research draft with positive mechanism checks and negative controls, not a submission claiming a deployed benchmark winner.

9 Conclusion

Witness-CL turns the question from “what should the agent remember?” into “which evidence rules out alternative behavior, and what justifies committing a change?” The finite-class answer is implementable and supports conditional retention and transfer. Its failure modes are equally concrete: misspecified abstractions can confidently fail, hidden changes break applicability, and fresh statistical evidence can arrive too slowly. The next scientific contribution must close those gaps on genuine stateful tasks under matched resources. The artifact makes that claim testable without claiming it has already been proved.

A Why unconditional guarantees need extra assumptions

Proposition 7 (An information barrier to universally safe improvement). *There exist two initially indistinguishable environments in which no learner can both improve and guarantee no expected degradation on every episode relative to a baseline, without an additional informative channel.*

Proof. In both environments, action a_0 returns $1/2$. In the first, action a_1 returns 1; in the second it returns 0. Before a_1 is tried, the observation history is identical. Choosing a_1 with probability $q > 0$ gives expected reward $1/2 - q/2$ in the second environment, below the baseline. Thus the guarantee forces $q = 0$. No distinguishing information arrives from a_0 , so induction forces continued baseline behavior in the first environment as well. Safe audit channels, structural knowledge, or a relaxation of the guarantee change the problem; retaining more of the same uninformative history does not. $\square$

Proposition 8 (Storage required by arbitrary exact retention). *A deterministic system that must later answer every query about N independently specified bits*

of experience exactly requires at least 2^N distinguishable persistent states, or N bits of total persistent storage.

Proof. There are 2^N possible histories. With fewer states, two distinct histories share a state by the pigeonhole principle. They differ at some bit. Given a query for that bit, the system receives the same retained state and query in both cases, so it must answer identically and be wrong in at least one. External memory counts as persistent state; structured compressible histories are a different assumption. $\square$

The result permits a bounded active context backed by growing storage. It prevents presenting an ever-growing disk archive as constant total memory or claiming arbitrary lifelong exact retention from a fixed-size summary.

B Reproduction and review checklist

Install the project with its test and analysis extras, then execute `make test`, `make experiment` and `make audit-power`. `make paper` regenerates numerical tables directly from recorded JSON and compiles this document. `make formal` requires the pinned Lean toolchain; a successful future run must replace the current unverified status only after reviewing actual compiler output.

The episode CSV records scenario, seed, method, public scope/version, input, action, reward, certificate flag and live-class size. Audit simulation output states its synthetic distribution and random seed. Independent reproductions should check the equality of full replay and Witness, witness reconstruction, negative-feedback semantics, stale-comparator rejection, hidden-change failures and missing-label rejection in online regression. New real-model results must include exact model identifiers, complete costs, native feedback-access checks and failed runs before any claim of public-benchmark superiority is made.

Authorship and disclosure. This is an AI-assisted draft prepared for Samuel Mausberg. Literature selection, proofs, implementation, data and scientific claims require author review and independent replication. No affiliation, funding award, accepted venue or DOI is asserted. The repository is distributed under the MIT license.

References

- [1] Parth Asawa, Christopher M. Glaze, Gabriel Orlanski, Ramya Ramakrishnan, Benji Xu, Asim Biswal, Vincent Sunn Chen, Frederic Sala, Matei Zaharia, Joseph E. Gonzalez. Continual Learning Bench: Evaluating Frontier AI Systems in Real-World Stateful Environments. 2026. <https://arxiv.org/abs/2606.05661>.
- [2] Yiheng Shu, Bernal Jiménez Gutiérrez, Saisri Padmaja Jonnalagedda, Yuguang Yao, Huan Sun, Yu Su. AgentCL: Toward Rigorous Evaluation of Continual Learning in Language Agents. 2026. <https://arxiv.org/abs/2606.02461>.
- [3] Qizheng Zhang, et al.. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. 2025. <https://arxiv.org/abs/2510.04618>.
- [4] Guanzhi Wang, et al.. Voyager: An Open-Ended Embodied Agent with Large Language Models. 2023. <https://arxiv.org/abs/2305.16291>.
- [5] Yufei He, et al.. EvoTest: Evolutionary Test-Time Learning for Self-Improving Agentic Systems. 2025. <https://arxiv.org/abs/2510.13220>.
- [6] Jun Nie, et al.. TTHE: Test-Time Harness Evolution. 2026. <https://arxiv.org/abs/2607.08124>.
- [7] Yiming Xiong, Shengran Hu, Jeff Clune. Learning to Continually Learn via Meta-learning Agentic Memory Designs. 2026. <https://arxiv.org/abs/2602.07755>.
- [8] Bojie Li. User as Code: Executable Memory for Personalized Agents. 2026. <https://arxiv.org/abs/2606.16707>.
- [9] Xinyu Guan, Qianyang Zhao, Yuming Deng. Decision-Aware Memory Cards: Counterfactual-Inspired Context Selection and Compression for Tool-Using LLM Agents. 2026. Version 3, September 4. <https://arxiv.org/abs/2606.08151>.
- [10] Lihong Li, Michael L. Littman, Thomas J. Walsh. Knows What It Knows: A Framework for Self-Aware Learning. In *Proceedings of the 25th International Conference on Machine Learning*, 2008. <https://doi.org/10.1145/1390156.1390228>.
- [11] Armando Solar-Lezama. Program Sketching. *International Journal on Software Tools for Technology Transfer*, 15:475–495, 2013. <https://doi.org/10.1007/s10009-012-0249-7>.
- [12] Yewen Pu, Zachery Miranda, Armando Solar-Lezama, Leslie Kaelbling. Selecting Representative Examples for Program Synthesis. In *Proceedings of the 35th International Conference on Machine Learning*, 2018. <https://proceedings.mlr.press/v80/pu18b.html>.
- [13] Abbas Kazerouni, Mohammad Ghavamzadeh, Yasin Abbasi-Yadkori, Benjamin Van Roy. Conservative Contextual Linear Bandits. 2017. <https://arxiv.org/abs/1611.06426>.
- [14] Steven R. Howard, Aaditya Ramdas, Jon McAuliffe, Jasjeet Sekhon. Time-uniform, Nonparametric, Nonasymptotic Confidence Sequences. *The Annals of Statistics*, 49(2):1055–1080, 2021. <https://arxiv.org/abs/1810.08240>.
- [15] James Kirkpatrick, et al.. Overcoming Catastrophic Forgetting in Neural Networks. 2016. <https://arxiv.org/abs/1612.00796>.
- [16] David Lopez-Paz, Marc’Aurelio Ranzato. Gradient Episodic Memory for Continual Learning. 2017. <https://arxiv.org/abs/1706.08840>.
- [17] Andrei A. Rusu, et al.. Progressive Neural Networks. 2016. <https://arxiv.org/abs/1606.04671>.
- [18] Arnub Tandon, et al.. End-to-End Test-Time Training for Long Context. 2025. <https://arxiv.org/abs/2512.23675>.